

Guía de Traspaso: PhoneFix Financial System

Este documento detalla el estado actual del proyecto 'PhoneFix Financial System', los cambios realizados, las credenciales relevantes y las recomendaciones para continuar el desarrollo.

1. Estado Actual del Proyecto

El proyecto es una aplicación web para la gestión financiera de PhoneFix, con funcionalidad multi-tienda (administrador y sucursal), generación de recibos bilingües en PDF y configuración de tienda. La aplicación está desplegada en Railway.

Funcionalidades implementadas exitosamente:

- Sistema multi-tienda con datos separados para administrador y sucursal.
- Página de configuración de tienda con información de contacto dinámica.
- Generación de recibos PDF bilingües (inglés/español) utilizando PDFKit.
- Diseño de recibos en blanco y negro con el logo 1+PhoneFix.
- Sección de garantía detallada en ambos idiomas.
- Autenticación basada en sesiones con `cookie-parser` para filtrado por tienda.
- Desplegado en Railway: <https://phoenix-financial-system-production.up.railway.app/>

Problemas identificados y abordados en esta sesión:

1. **Error al agregar reparaciones:** Se identificó un error en el cálculo de la ganancia en el endpoint `createRepair` y `addRepairParts` que causaba problemas al añadir reparaciones. **(Corregido)**
2. **Filtrado de configuración de tienda:** Se identificó que la sucursal veía la configuración del administrador debido a un uso incorrecto del endpoint `user.me`. **(Corregido)**
3. **Nueva sección "Servidor" pendiente:** Creación de una nueva sección para servicios de desbloqueo/software con integración de API. (Pendiente de implementación)

2. Credenciales y Accesos

Repositorio de GitHub:

- <https://github.com/gghhff/Phoenix-Financial-System>

API Key para la sección "Servidor" (Desbloques/Software):

- 6T0-V56-CPP-IGB-K0Q-54F-9TL-1WB

Nota Importante sobre Credenciales Sensibles:

Las credenciales de acceso a la base de datos (PostgreSQL en Railway), cuentas de usuario (admin/sucursal) y cualquier otra clave de API que no sea la mencionada anteriormente, **no pueden ser proporcionadas por este agente** por motivos de seguridad. Debes obtener estas credenciales directamente del propietario del proyecto o de la configuración de tu entorno en Railway.

3. Resumen de Cambios Realizados en esta Sesión

Durante esta sesión, se realizaron las siguientes modificaciones en el código:

- **server/routers.ts :**
 - Se agregó un nuevo router `user` con un endpoint `me` (`user: router({ me: publicProcedure.query(opts => opts.ctx.user), })`) para asegurar la compatibilidad con el frontend que esperaba `trpc.user.me` .
- **server/db.ts :**
 - Se corrigió el cálculo de la ganancia en la función `createRepair` . Anteriormente, solo se restaba el `costoPartes` del `precioTotal` . Ahora, la fórmula correcta es `precioTotal - precioManoObra - costoPartes` .
 - Se corrigió el cálculo de la ganancia en la función `addRepairParts` para que también considere el `precioManoObra` al actualizar la ganancia de una reparación cuando se añaden partes.

Estos cambios se han subido al repositorio de GitHub y deberían haberse desplegado automáticamente en Railway.

4. Punto de Pausa y Recomendaciones para Continuar

Punto de Pausa:

La tarea se pausó después de corregir los errores de cálculo de ganancia en las reparaciones y de asegurar que el endpoint `user.me` esté disponible. Los cambios han sido subidos a GitHub y el respaldo del sandbox ha sido creado.

Recomendaciones para Continuar:

1. **Verificar las reparaciones:** A pesar de las correcciones, el usuario reportó que las reparaciones aún no se pueden ingresar. Es crucial realizar un **diagnóstico profundo** para entender la causa raíz de este problema persistente. Esto podría implicar:
 - Revisar los logs de Railway para errores específicos.

- Depurar el proceso de creación de reparaciones tanto en el frontend como en el backend.
 - Verificar la integridad de la base de datos y los esquemas (`drizzle/schema.ts`).
 - Asegurarse de que los tipos de datos (`z.string()` , `z.number()`) en los inputs de tRPC coincidan con los tipos esperados en la base de datos y en las funciones de `db.ts` .
 - Probar la creación de reparaciones con y sin partes de inventario, y sin partes manuales, para aislar el problema.
2. **Implementar la sección "Servidor"**: Una vez que los problemas de reparación estén resueltos, el siguiente paso es implementar la nueva sección "Servidor" siguiendo el plan detallado en `/home/ubuntu/PLAN_API_DESBLOQUEOS.md` . Recuerda:
- La API solo debe ejecutarse en esta nueva sección y no debe alterar el sistema existente.
 - La sucursal debe ver un mensaje de "En construcción".
 - El administrador debe tener funcionalidad completa.
3. **Despliegue y Pruebas**: Después de cada fase de implementación, realizar un despliegue en Railway y pruebas exhaustivas para asegurar que todo funciona como se espera y que no se introducen nuevas regresiones.

5. Enlace de Respaldo del Sandbox

Se ha generado un archivo comprimido (`.tar.gz`) del directorio del proyecto (excluyendo `node_modules`) para tu conveniencia. Puedes descargarlo desde el siguiente enlace:

- https://files.manuscdn.com/user_upload_by_module/session_file/310519663238741508/JSBxnyeEwqHfDUVD.gz

Este archivo contiene el estado del código en el momento de la pausa, incluyendo las últimas correcciones.